

DD2440 Advanced Algorithms
Fall 2026

Lecture 4: Optimization of submodular functions

Ioana Bercea (slides mainly from Aris)

2026-09-21

Agenda

Submodular optimization

Maximization of submodular functions with cardinality constraints

Maximization of submodular functions under a matroid constraint

Maximization of non-monotone submodular functions

Max-sum diversification

Reminders

- ▶ group assignment 1 started at 8:00
 - ▶ please check that you can submit as a group!
 - ▶ you do not need to do the peer reviews
- ▶ exercise sessions this week (check Canvas for time)

combinatorial optimization

- many problems of interest are formalized as problems in **combinatorial optimization**
 - *set cover, maximum coverage, vertex cover, clique, graph matching, min cut, max cut, minimum spanning tree, minimum Steiner tree, TSP, ...*
- all these problems can be written in the form

$$\begin{aligned} &\max \{f(S) : S \in \mathcal{F}\}, \text{ or} \\ &\min \{f(S) : S \in \mathcal{F}\}, \end{aligned}$$

where $\mathcal{F}$ is a **discrete set of feasible solutions**

combinatorial optimization

- many problems of interest are formalized as problems in **combinatorial optimization**
 - *set cover, maximum coverage, vertex cover, clique, graph matching, min cut, max cut, minimum spanning tree, minimum Steiner tree, TSP, ...*
- all these problems can be written in the form

$$\begin{aligned} &\max \{f(S) : S \in \mathcal{F}\}, \text{ or} \\ &\min \{f(S) : S \in \mathcal{F}\}, \end{aligned}$$

where $\mathcal{F}$ is a **discrete set of feasible solutions**

- we can study these problems
 - **individually**, trying to solve one at a time

combinatorial optimization

- many problems of interest are formalized as problems in **combinatorial optimization**
 - *set cover, maximum coverage, vertex cover, clique, graph matching, min cut, max cut, minimum spanning tree, minimum Steiner tree, TSP, ...*
- all these problems can be written in the form

$$\begin{aligned} &\max \{f(S) : S \in \mathcal{F}\}, \text{ or} \\ &\min \{f(S) : S \in \mathcal{F}\}, \end{aligned}$$

where $\mathcal{F}$ is a **discrete set of feasible solutions**

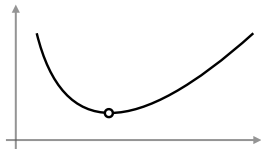
- we can study these problems
 - **individually**, trying to solve one at a time
 - identify **properties** of f and $\mathcal{F}$ that make them **tractable**

taking inspiration from continuous optimization

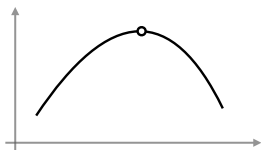
- what are such properties in continuous optimization?

taking inspiration from continuous optimization

- what are such properties in continuous optimization?



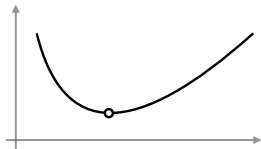
- a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ can be minimized efficiently,
if it is convex



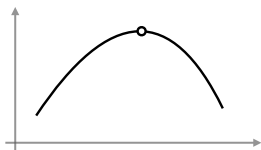
- a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ can be maximized efficiently,
if it is concave

taking inspiration from continuous optimization

- what are such properties in continuous optimization?



- a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ can be **minimized** efficiently,
if it is **convex**



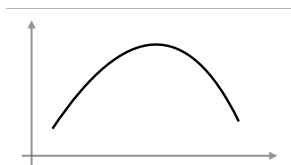
- a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ can be **maximized** efficiently,
if it is **concave**

- what would be the discrete analog of **convexity** and/or **concavity**?

we will be looking at **set functions** $f : \{0, 1\}^n \rightarrow \mathbb{R}$

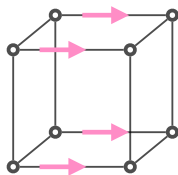
from concavity to submodularity

concavity



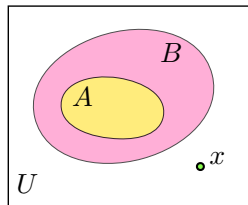
- function $f : \mathbb{R} \rightarrow \mathbb{R}$ is **concave**
if the **derivative** $f'(x)$ is **non-increasing** in x

submodularity



- function $f : \{0, 1\}^n \rightarrow \mathbb{R}$ is **submodular**
if for all $i \in [n]$, the **discrete derivative**
 $\partial_i f(x) = f(x + e_i) - f(x)$ is **non-increasing** in x

definition of submodular set functions



– consider a ground set U

– a function $f : 2^U \rightarrow \mathbb{R}$ is **submodular** if

$$f(A \cup \{x\}) - f(A) \geq f(B \cup \{x\}) - f(B)$$

for all $A \subseteq B \subseteq U$ and $x \in U \setminus B$

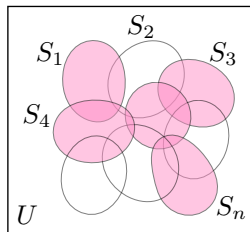
(also known as “**diminishing returns**” property)

– an equivalent definition is to require

$$f(A) + f(B) \geq f(A \cup B) + f(A \cap B)$$

for all $A, B \subseteq U$

example: coverage



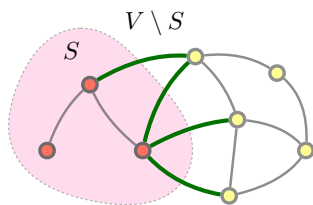
- consider a ground set U , with weights $w : U \rightarrow \mathbb{R}_+$, and $S_1, \dots, S_n$ subsets of U
- function $f : 2^{\{1, \dots, n\}} \rightarrow \mathbb{R}_+$
(defined over subsets of indices in $[n]$)
- **coverage**: the number of items in U contained in the sets specified by A

$$f(A) = |\cup_{i \in A} S_i|$$

- **weighted coverage**: the total weight of the items in U contained in the sets specified by A

$$f(A) = \sum_{x \in \cup_{i \in A} S_i} w(x)$$

example: graph cut



- consider an undirected graph $G = (V, E)$
- cut function $f : 2^V \rightarrow R_+$
(defined over subsets of vertices in V)
the number of edges from S to $V \setminus S$

$$f(S) = |E(S, V \setminus S)|$$

- can also define weighted cut, for graphs with edge weights

submodular set functions

may or not satisfy the following properties

- non-negativity : $f(A) \geq 0$ for all $A \subseteq U$
- monotonicity : $f(A) \leq f(B)$ for all $A \subseteq B \subseteq U$
- symmetry : $f(A) = f(U \setminus A)$ for all $A \subseteq U$

quiz

does non-negativity, monotonicity, and symmetry hold for coverage and graph cut?

coverage in set systems

cut function in undirected graphs and hypergraphs

cut function in directed graphs

quiz

does non-negativity, monotonicity, and symmetry hold for coverage and graph cut?

coverage in set systems

$\Rightarrow$ non-negative and monotone (but not symmetric)

cut function in undirected graphs and hypergraphs

cut function in directed graphs

quiz

does non-negativity, monotonicity, and symmetry hold for coverage and graph cut?

coverage in set systems

$\Rightarrow$ non-negative and monotone (but not symmetric)

cut function in undirected graphs and hypergraphs

$\Rightarrow$ non-negative and symmetric (but non monotone)

cut function in directed graphs

quiz

does non-negativity, monotonicity, and symmetry hold for coverage and graph cut?

coverage in set systems

$\Rightarrow$ non-negative and monotone (but not symmetric)

cut function in undirected graphs and hypergraphs

$\Rightarrow$ non-negative and symmetric (but non monotone)

cut function in directed graphs, btw, is it submodular?

quiz

does non-negativity, monotonicity, and symmetry hold for coverage and graph cut?

coverage in set systems

$\Rightarrow$ non-negative and monotone (but not symmetric)

cut function in undirected graphs and hypergraphs

$\Rightarrow$ non-negative and symmetric (but non monotone)

cut function in directed graphs, btw, is it submodular?

$\Rightarrow$ submodular and non-negative (but neither monotone nor symmetric)

concave or convex?

so, is submodularity more like concavity or convexity?

- argument for concavity:

definition looks more like concavity — non-increasing discrete derivatives

- argument for convexity:

submodularity seems to be more useful for minimization than maximization

concave or convex?

so, is submodularity more like concavity or convexity?

- argument for concavity:

definition looks more like concavity — non-increasing discrete derivatives

- argument for convexity:

submodularity seems to be more useful for minimization than maximization

Theorem 1.

*There is an algorithm that computes the **minimum** of any submodular function $f : \{0, 1\}^n \rightarrow \mathbb{R}$ in **polynomial time**, using oracle queries of the form $f(S) = ?$.*

concave or convex?

so, is submodularity more like concavity or convexity?

- argument for concavity:

definition looks more like concavity — non-increasing discrete derivatives

- argument for convexity:

submodularity seems to be more useful for minimization than maximization

Theorem 1.

*There is an algorithm that computes the **minimum** of any submodular function $f : \{0, 1\}^n \rightarrow \mathbb{R}$ in **polynomial time**, using oracle queries of the form $f(S) = ?$.*

on the other hand, **maximizing** a submodular function (e.g., max cut) is **NP-hard**

convex aspects of submodular functions

why is it possible to minimize submodular functions?

- the combinatorial algorithms are sophisticated ... rely on the **Lovász extension**

submodular function

$$f : \{0, 1\}^n \rightarrow \mathbb{R}$$



convex continuous
function

$$f^L : [0, 1]^n \rightarrow \mathbb{R}$$

- if f is submodular, then f^L is convex
- therefore, f^L can be minimized efficiently using the **ellipsoid algorithm**
- a minimizer of $f^L(x)$ can be converted into a minimizer of $f(S)$

Agenda

Submodular optimization

Maximization of submodular functions with cardinality constraints

Maximization of submodular functions under a matroid constraint

Maximization of non-monotone submodular functions

Max-sum diversification

maximization of submodular functions

- given submodular function $f : 2^U \rightarrow \mathbb{R}$
find $S \subseteq X$ to maximize $f(S)$
subject to constraints
- value-oracle model
- generalizes many interesting problems **NP**-hard problems
 - e.g., set cover, max cut, vertex cover, etc.
- comes up naturally in many settings satisfying the diminishing return property
 - e.g., sensor placement, viral marketing, welfare maximization, etc.
 - in such settings monotonicity typically also holds, $A \subseteq B \Rightarrow f(A) \leq f(B)$

monotone functions

we consider maximizing submodular and monotone functions

- in the unconstrained case, ...

monotone functions

we consider maximizing submodular and monotone functions

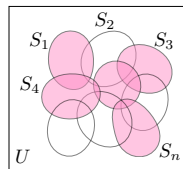
- in the unconstrained case, ... $f(U)$ is a trivial maximizer

monotone functions

we consider maximizing submodular and monotone functions

- in the unconstrained case, ... $f(U)$ is a trivial maximizer
- the constrained case is more interesting

a common setting is submodular maximization with cardinality constraints
that is, find $S \subseteq U$ subject to $|S| \leq k$ that maximizes $f(S)$



MAX k -COVER is a special case

greedy gives $(1 - 1/e)$ approximation [Nemhauser et al. '78]

no better approximation unless $P = NP$ [Feige '98]

the greedy algorithm

1. $S \leftarrow \emptyset$
2. while $|S| < k$
3. $i \leftarrow \arg \max_j f(S \cup \{j\})$
4. $S \leftarrow S \cup \{i\}$
5. return S

intuition for the analysis

- ▶ every time we add a new element to S , we get closer to **Opt**
- ▶ think of minimizing **the gap** between $f(S)$ and **Opt** every time step

intuition for the analysis

- ▶ every time we add a new element to S , we get closer to **Opt**
- ▶ think of minimizing **the gap** between $f(S)$ and **Opt** every time step
- ▶ the first element gets **Opt**/ k , so the gap is $(1 - 1/k) \cdot \mathbf{Opt}$
- ▶ **diminishing returns**: the next element gets $1/k$ of the remaining gap (nontrivial)
- ▶ so we are left with a gap of $(1 - 1/k)^2 \cdot \mathbf{Opt}$ after we add two elements

intuition for the analysis

- ▶ every time we add a new element to S , we get closer to **Opt**
- ▶ think of minimizing **the gap** between $f(S)$ and **Opt** every time step
- ▶ the first element gets **Opt**/ k , so the gap is $(1 - 1/k) \cdot \mathbf{Opt}$
- ▶ **diminishing returns**: the next element gets $1/k$ of the remaining gap (nontrivial)
- ▶ so we are left with a gap of $(1 - 1/k)^2 \cdot \mathbf{Opt}$ after we add two elements
- ▶ after adding i elements, we are left with a gap of $(1 - 1/k)^i \cdot \mathbf{Opt}$

intuition for the analysis

- ▶ every time we add a new element to S , we get closer to **Opt**
- ▶ think of minimizing **the gap** between $f(S)$ and **Opt** every time step
- ▶ the first element gets **Opt**/ k , so the gap is $(1 - 1/k) \cdot \mathbf{Opt}$
- ▶ **diminishing returns**: the next element gets $1/k$ of the remaining gap (nontrivial)
- ▶ so we are left with a gap of $(1 - 1/k)^2 \cdot \mathbf{Opt}$ after we add two elements
- ▶ after adding i elements, we are left with a gap of $(1 - 1/k)^i \cdot \mathbf{Opt}$
- ▶ for k large enough, the gap becomes $1/e \approx 0.36$

more intuition for the analysis

- ▶ think of minimizing **the gap** between $f(S)$ and **Opt** every time step
- ▶ every element we add, we decrease the remaining gap by a factor of $1/k$:

more intuition for the analysis

- ▶ think of minimizing **the gap** between $f(S)$ and **Opt** every time step
- ▶ every element we add, we decrease the remaining gap by a factor of $1/k$:
- ▶ define S_i be first i elements picked by the greedy, let S^* be some **Opt** solution
- ▶ i.e., we want to show that:

$$f(S_i) - f(S_{i-1}) \geq \frac{1}{k} \cdot (f(S^*) - f(S_{i-1}))$$

more intuition for the analysis

- ▶ think of minimizing **the gap** between $f(S)$ and **Opt** every time step
- ▶ every element we add, we decrease the remaining gap by a factor of $1/k$:
- ▶ define S_i be first i elements picked by the greedy, let S^* be some **Opt** solution
- ▶ i.e., we want to show that:

$$f(S_i) - f(S_{i-1}) \geq \frac{1}{k} \cdot (f(S^*) - f(S_{i-1}))$$

- ▶ why would this be true?

more intuition for the analysis

- ▶ think of minimizing **the gap** between $f(S)$ and **Opt** every time step
- ▶ every element we add, we decrease the remaining gap by a factor of $1/k$:
- ▶ define S_i be first i elements picked by the greedy, let S^* be some **Opt** solution
- ▶ i.e., we want to show that:

$$f(S_i) - f(S_{i-1}) \geq \frac{1}{k} \cdot (f(S^*) - f(S_{i-1}))$$

- ▶ why would this be true?
- ▶ intuitively: because we always pick the best element in each round, and this gives more gain than any of the k elements in **Opt**

analysis of the greedy (1)

- recall that we want to show that:

$$f(S_i) - f(S_{i-1}) \geq \frac{1}{k} \cdot (f(S^*) - f(S_{i-1}))$$

analysis of the greedy (1)

- recall that we want to show that:

$$f(S_i) - f(S_{i-1}) \geq \frac{1}{k} \cdot (f(S^*) - f(S_{i-1}))$$

- we write $f_S(e) = f(S \cup \{e\}) - f(S)$ for the **marginal gain** of adding e to S
- note that, in each step, we try to pick the element that maximizes the marginal gain

analysis of the greedy (1)

- ▶ recall that we want to show that:

$$f(S_i) - f(S_{i-1}) \geq \frac{1}{k} \cdot (f(S^*) - f(S_{i-1}))$$

- ▶ we write $f_S(e) = f(S \cup \{e\}) - f(S)$ for the **marginal gain** of adding e to S
- ▶ note that, in each step, we try to pick the element that maximizes the marginal gain
- ▶ what this means is that for any $e \notin S_{i-1}$:

$$f(S_i) - f(S_{i-1}) \geq f_{S_{i-1}}(e)$$

analysis of the greedy (1)

- ▶ recall that we want to show that:

$$f(S_i) - f(S_{i-1}) \geq \frac{1}{k} \cdot (f(S^*) - f(S_{i-1}))$$

- ▶ we write $f_S(e) = f(S \cup \{e\}) - f(S)$ for the **marginal gain** of adding e to S
- ▶ note that, in each step, we try to pick the element that maximizes the marginal gain
- ▶ what this means is that for any $e \notin S_{i-1}$:

$$f(S_i) - f(S_{i-1}) \geq f_{S_{i-1}}(e)$$

- ▶ in particular, this holds for all $e \in S^* \setminus S_{i-1}$

analysis of the greedy (1)

- ▶ recall that we want to show that:

$$f(S_i) - f(S_{i-1}) \geq \frac{1}{k} \cdot (f(S^*) - f(S_{i-1}))$$

- ▶ we write $f_S(e) = f(S \cup \{e\}) - f(S)$ for the **marginal gain** of adding e to S
- ▶ note that, in each step, we try to pick the element that maximizes the marginal gain
- ▶ what this means is that for any $e \notin S_{i-1}$:

$$f(S_i) - f(S_{i-1}) \geq f_{S_{i-1}}(e)$$

- ▶ in particular, this holds for all $e \in S^* \setminus S_{i-1}$
- ▶ so by averaging:

$$f(S_i) - f(S_{i-1}) \geq \frac{1}{k} \cdot \sum_{e \in S^* \setminus S_{i-1}} f_{S_{i-1}}(e)$$

analysis of the greedy (2)

- note that submodularity implies that marginal gain decreases: for $A \subseteq B$ and $x \notin B$:

$$f_A(x) \geq f_B(x)$$

analysis of the greedy (2)

- ▶ note that submodularity implies that marginal gain decreases: for $A \subseteq B$ and $x \notin B$:

$$f_A(x) \geq f_B(x)$$

- ▶ consider any ordering of the elements in the optimal solution S^*
- ▶ and let S_e^* be the elements in the optimal that precede $e \in S^*$ in that order

analysis of the greedy (2)

- ▶ note that submodularity implies that marginal gain decreases: for $A \subseteq B$ and $x \notin B$:

$$f_A(x) \geq f_B(x)$$

- ▶ consider any ordering of the elements in the optimal solution S^*
- ▶ and let S_e^* be the elements in the optimal that precede $e \in S^*$ in that order
- ▶ in particular, we want to lower bound $f_{S_{i-1}}(e)$

analysis of the greedy (2)

- ▶ note that submodularity implies that marginal gain decreases: for $A \subseteq B$ and $x \notin B$:

$$f_A(x) \geq f_B(x)$$

- ▶ consider any ordering of the elements in the optimal solution S^*
- ▶ and let S_e^* be the elements in the optimal that precede $e \in S^*$ in that order
- ▶ in particular, we want to lower bound $f_{S_{i-1}}(e)$
- ▶ bound it by $f_{S_{i-1} \cup S_e^*}(e)$:

$$f_{S_{i-1}}(e) \geq f_{S_{i-1} \cup S_e^*}(e) .$$

analysis of the greedy (2)

- ▶ note that submodularity implies that marginal gain decreases: for $A \subseteq B$ and $x \notin B$:

$$f_A(x) \geq f_B(x)$$

- ▶ consider any ordering of the elements in the optimal solution S^*
- ▶ and let S_e^* be the elements in the optimal that precede $e \in S^*$ in that order
- ▶ in particular, we want to lower bound $f_{S_{i-1}}(e)$
- ▶ bound it by $f_{S_{i-1} \cup S_e^*}(e)$:

$$f_{S_{i-1}}(e) \geq f_{S_{i-1} \cup S_e^*}(e) .$$

- ▶ the trick:

$$\sum_{e \in S^* \setminus S_{i-1}} f_{S_{i-1} \cup S_e^*}(e) = f(S_{i-1} \cup S^*) - f(S_{i-1})$$

putting it all together

► for all $i \leq k$ it holds

$$f(S^*) \leq f(S^* \cup S_{i-1}) \quad (\text{monotonicity})$$

$$= f(S_{i-1}) + \sum_{e \in S^* \setminus S_{i-1}} f_{S_{i-1} \cup S_e^*}(e)$$

$$\leq f(S_{i-1}) + \sum_{e \in S^* \setminus S_{i-1}} f_{S_{i-1}}(e) \quad (\text{submodularity})$$

$$\leq f(S_{i-1}) + \sum_{e \in S^* \setminus S_{i-1}} (f(S_i) - f(S_{i-1})) \quad (\text{greediness})$$

$$\leq f(S_{i-1}) + k(f(S_i) - f(S_{i-1})) \quad (|S^*| \leq k)$$

putting it all together (cont'ed)

we showed

$$f(S^*) \leq f(S_{i-1}) + k(f(S_i) - f(S_{i-1}))$$

it follows

$$f(S_i) - f(S^*) \geq \frac{k-1}{k} (f(S_{i-1}) - f(S^*))$$

which by induction implies

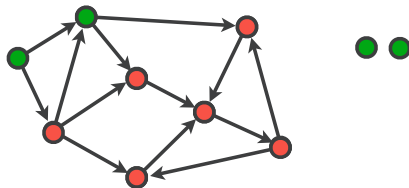
$$f(S_i) \geq \left(1 - (1 - 1/k)^i\right) f(S^*)$$

and so

$$f(S_{greedy}) = f(S_k) \geq \left(1 - (1 - 1/k)^k\right) f(S^*) \geq \left(1 - \frac{1}{e}\right) f(S^*)$$

widely applicable in data mining

- **example** : maximize the **spread of influence** in social networks Kempe et al.'03
- assume that an action is spread in a social network
- assume a spreading model such as **independent cascade**
- find a set of **k** initial seeds to maximize the spread
- spreading model is randomized, so we want to maximize **expected spread**



Agenda

Submodular optimization

Maximization of submodular functions with cardinality constraints

Maximization of submodular functions under a matroid constraint

Maximization of non-monotone submodular functions

Max-sum diversification

submodular maximization under a matroid constraint

[Nemhauser et al. '78] considered a more general problem :

given a monotone submodular function f and matroid $\mathcal{M} = (U, \mathcal{I})$

find $S \in \mathcal{I}$ maximizing $f(S)$

they showed that the greedy algorithm gives a $1/2$ -approximation for the problem $\max\{f(S) : S \in \mathcal{I}\}$

and more generally (intersection of k matroids) :

$\frac{1}{k+1}$ -approximation for the problem $\max\{f(S) : S \in \mathcal{I}_1 \cap \dots \cap \mathcal{I}_k\}$

but what are matroids and what can be modeled using a matroid constraint?

matroids

definition :

a matroid $\mathcal{M} = (U, \mathcal{I})$ on U is a system of independent sets $\mathcal{I} \subseteq 2^U$ satisfying

1. for all $B \in \mathcal{I}$, $A \subset B \Rightarrow A \in \mathcal{I}$
2. for all $A, B \in \mathcal{I}$, $|A| < |B| \Rightarrow$ there exists $x \in B \setminus A$ such that $A \cup \{x\} \in \mathcal{I}$

matroids

definition :

a matroid $\mathcal{M} = (U, \mathcal{I})$ on U is a system of independent sets $\mathcal{I} \subseteq 2^U$ satisfying

1. for all $B \in \mathcal{I}$, $A \subset B \Rightarrow A \in \mathcal{I}$
2. for all $A, B \in \mathcal{I}$, $|A| < |B| \Rightarrow$ there exists $x \in B \setminus A$ such that $A \cup \{x\} \in \mathcal{I}$

examples :

- uniform matroid: $\{S \in 2^U : |S| \leq k\}$
- partition matroid: U is partitioned in disjoint groups $G_1, \dots, G_t$, and the independent sets contain at most k_i items from group G_i
- cycle matroid: U is the set of edges of a graph G , and the independent sets are subsets of edges contain no cycle (i.e., forests)

matroids - intuition

definition :

a matroid $\mathcal{M} = (U, \mathcal{I})$ on U is a system of independent sets $\mathcal{I} \subseteq 2^U$ satisfying

1. for all $B \in \mathcal{I}$, $A \subset B \Rightarrow A \in \mathcal{I}$
2. for all $A, B \in \mathcal{I}$, $|A| < |B| \Rightarrow$ there exists $x \in B \setminus A$ such that $A \cup \{x\} \in \mathcal{I}$

matroids - intuition

definition :

a matroid $\mathcal{M} = (U, \mathcal{I})$ on U is a system of independent sets $\mathcal{I} \subseteq 2^U$ satisfying

1. for all $B \in \mathcal{I}$, $A \subset B \Rightarrow A \in \mathcal{I}$
2. for all $A, B \in \mathcal{I}$, $|A| < |B| \Rightarrow$ there exists $x \in B \setminus A$ such that $A \cup \{x\} \in \mathcal{I}$

meant to model when a greedy algorithm yields the exact optimal solution rather than just an approximation

matroids - intuition

definition :

a matroid $\mathcal{M} = (U, \mathcal{I})$ on U is a system of independent sets $\mathcal{I} \subseteq 2^U$ satisfying

1. for all $B \in \mathcal{I}$, $A \subset B \Rightarrow A \in \mathcal{I}$
2. for all $A, B \in \mathcal{I}$, $|A| < |B| \Rightarrow$ there exists $x \in B \setminus A$ such that $A \cup \{x\} \in \mathcal{I}$

meant to model when a greedy algorithm yields the exact optimal solution rather than just an approximation

forest or linearly independent vectors :

- ▶ a single edge is independent
- ▶ if you have a forest, removing an edge keeps it a forest
- ▶ if you have a larger forest, you can always "steal" an edge to grow the smaller forest

matroids - further intuition

definition :

a matroid $\mathcal{M} = (U, \mathcal{I})$ on U is a system of independent sets $\mathcal{I} \subseteq 2^U$ satisfying

1. for all $B \in \mathcal{I}$, $A \subset B \Rightarrow A \in \mathcal{I}$
2. for all $A, B \in \mathcal{I}$, $|A| < |B| \Rightarrow$ there exists $x \in B \setminus A$ such that $A \cup \{x\} \in \mathcal{I}$

matroids - further intuition

definition :

a matroid $\mathcal{M} = (U, \mathcal{I})$ on U is a system of independent sets $\mathcal{I} \subseteq 2^U$ satisfying

1. for all $B \in \mathcal{I}$, $A \subset B \Rightarrow A \in \mathcal{I}$
2. for all $A, B \in \mathcal{I}$, $|A| < |B| \Rightarrow$ there exists $x \in B \setminus A$ such that $A \cup \{x\} \in \mathcal{I}$

maximal forest of a weighted connected graph = spanning tree

matroids - further intuition

definition :

a matroid $\mathcal{M} = (U, \mathcal{I})$ on U is a system of independent sets $\mathcal{I} \subseteq 2^U$ satisfying

1. for all $B \in \mathcal{I}$, $A \subset B \Rightarrow A \in \mathcal{I}$
2. for all $A, B \in \mathcal{I}$, $|A| < |B| \Rightarrow$ there exists $x \in B \setminus A$ such that $A \cup \{x\} \in \mathcal{I}$

maximal forest of a weighted connected graph = spanning tree

Greedy always finds **Opt** (Kruskal's algorithm)

1. Sort all edges by weight
2. Pick the cheapest edge that doesn't create a cycle
3. Repeat until you have $V - 1$ edges

submodular welfare maximization

given : set $U = \{u_1, \dots, u_m\}$ of m items, and set $P = \{p_1, \dots, p_n\}$ of n players
player p_i has submodular valuation function $f_i : 2^m \rightarrow \mathbb{R}$

welfare maximization : find partition $U = S_1 \cup \dots \cup S_n$ to maximize $\sum_i^n f_i(S_i)$

submodular welfare maximization

given : set $U = \{u_1, \dots, u_m\}$ of m items, and set $P = \{p_1, \dots, p_n\}$ of n players
player p_i has submodular valuation function $f_i : 2^m \rightarrow \mathbb{R}$

welfare maximization : find partition $U = S_1 \cup \dots \cup S_n$ to maximize $\sum_i^n f_i(S_i)$

how to solve it?

submodular welfare maximization

given : set $U = \{u_1, \dots, u_m\}$ of m items, and set $P = \{p_1, \dots, p_n\}$ of n players
player p_i has submodular valuation function $f_i : 2^m \rightarrow \mathbb{R}$

welfare maximization : find partition $U = S_1 \cup \dots \cup S_n$ to maximize $\sum_i^n f_i(S_i)$

reduction :

create n clones of each item, one clone for each player

let W be the set of all items together with their clones

let P_i be the clones of items corresponding to player p_i

let Q_j be the clones of item e_j

welfare maximization problem is equivalent to selecting subset $S \subseteq W$

so as to maximize the submodular function $f(S) = \sum_i^n f_i(S \cap P_i)$

with partition matroid constraint $\mathcal{I} = \{S \subseteq W : |S \cap Q_j| \leq 1, \text{ for all } j\}$

submodular welfare maximization

given : set $U = \{u_1, \dots, u_m\}$ of m items, and set $P = \{p_1, \dots, p_n\}$ of n players
player p_i has submodular valuation function $f_i : 2^m \rightarrow \mathbb{R}$

welfare maximization : find partition $U = S_1 \cup \dots \cup S_n$ to maximize $\sum_i^n f_i(S_i)$

create n clones of each item, one clone for each player

let W be the set of all items together with their clones

let P_i be the clones of items corresponding to player p_i

let Q_j be the clones of item e_j

welfare maximization problem is equivalent to selecting subset $S \subseteq W$

so as to maximize the submodular function $f(S) = \sum_i^n f_i(S \cap P_i)$

with partition matroid constraint $\mathcal{I} = \{S \subseteq W : |S \cap Q_j| \leq 1, \text{ for all } j\}$

greedy gives $\frac{1}{2}$ -approximation

Agenda

Submodular optimization

Maximization of submodular functions with cardinality constraints

Maximization of submodular functions under a matroid constraint

Maximization of non-monotone submodular functions

Max-sum diversification

non-monotone functions

we consider maximizing submodular non-monotone functions

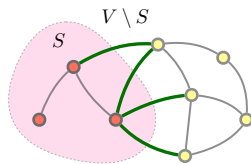
- unconstrained version becomes interesting

that is, find $S \subseteq X$ to maximize $f(S)$

non-monotone functions

we consider maximizing submodular non-monotone functions

- unconstrained version becomes interesting
that is, find $S \subseteq X$ to maximize $f(S)$
- generalizes the MAX-CUT problem
- what do we know about the approximation of MAX-CUT?



non-monotone functions

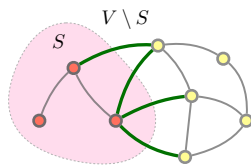
we consider maximizing submodular non-monotone functions

- unconstrained version becomes interesting
that is, find $S \subseteq X$ to maximize $f(S)$
- generalizes the MAX-CUT problem
- what do we know about the approximation of MAX-CUT?

greedy gives $1/2$

0.878 by SDP relaxation

0.53 by a spectral approach



Goeman and Williamson '95

Trevisan '12

the unconstrained problem

[Feige, Mirrokni, and Vondrák '11]

- first constant-factor approximations for non-negative submodular functions
- $1/2$ approximation for symmetric functions
- $2/5 = 0.4$ approximation for the non-negative functions
- lower bound: better than $1/2$ approximation requires exponential number of value queries
- random set selection algorithm (might discuss later)
- deterministic local search: instead of greedy for monotone

the unconstrained problem

[Feige, Mirrokni, and Vondrák '11]

local search

- initialize S to best singleton
- S = local optimum (add or delete elements)
- return the best of S and $U \setminus S$

$1/3$ approximation for non-negative functions

$1/2$ approximation for non-negative symmetric functions

analysis of local search

lemma: if S is a local optimum then $f(S) \geq f(T)$ for all $S \subseteq T$ and $T \subseteq S$

analysis of local search

lemma: if S is a local optimum then $f(S) \geq f(T)$ for all $S \subseteq T$ and $T \subseteq S$

proof

take $S \subseteq T$ and consider $S = X_0 \subseteq \dots \subseteq X_\ell = T$

by submodularity and local optimality

$$0 \geq f(S \cup \{x_i\}) - f(S) \geq f(X_i) - f(X_{i-1})$$

summing up gives $0 \geq f(X_\ell) - f(X_0)$ or $f(S) \geq f(T)$

analysis of local search

lemma: if S is a local optimum then $f(S) \geq f(T)$ for all $S \subseteq T$ and $T \subseteq S$

proof

take $S \subseteq T$ and consider $S = X_0 \subseteq \dots \subseteq X_\ell = T$

by submodularity and local optimality

$$0 \geq f(S \cup \{x_i\}) - f(S) \geq f(X_i) - f(X_{i-1})$$

summing up gives $0 \geq f(X_\ell) - f(X_0)$ or $f(S) \geq f(T)$

corollary

for optimum S^* and local optimum S it is $f(S) \geq f(S \cup S^*)$ and $f(S) \geq f(S \cap S^*)$

analysis of local search (cont'd)

- we just argued

$$f(S) \geq f(S \cup S^*) \quad \text{and} \quad f(S) \geq f(S \cap S^*)$$

- by submodularity and non-negativity

$$f(S \cup S^*) + f(U \setminus S) \geq f(S^* \setminus S) + f(U) \geq f(S^* \setminus S)$$

$$f(S \cap S^*) + f(S^* \setminus S) \geq f(S^*) + f(\emptyset) \geq f(S^*)$$

analysis of local search (cont'ed)

- we just argued

$$f(S) \geq f(S \cup S^*) \quad \text{and} \quad f(S) \geq f(S \cap S^*)$$

- by submodularity and non-negativity

$$f(S \cup S^*) + f(U \setminus S) \geq f(S^* \setminus S) + f(U) \geq f(S^* \setminus S)$$

$$f(S \cap S^*) + f(S^* \setminus S) \geq f(S^*) + f(\emptyset) \geq f(S^*)$$

- combining we get

$$2f(S) + f(U \setminus S) \geq f(S^*)$$

analysis of local search (cont'ed)

- we just argued

$$f(S) \geq f(S \cup S^*) \quad \text{and} \quad f(S) \geq f(S \cap S^*)$$

- by submodularity and non-negativity

$$f(S \cup S^*) + f(U \setminus S) \geq f(S^* \setminus S) + f(U) \geq f(S^* \setminus S)$$

$$f(S \cap S^*) + f(S^* \setminus S) \geq f(S^*) + f(\emptyset) \geq f(S^*)$$

- combining we get

$$2f(S) + f(U \setminus S) \geq f(S^*)$$

- and so

$$\max\{f(S), f(U \setminus S)\} \geq \frac{1}{3}f(S^*)$$

the unconstrained problem

can we do better than $1/3$ for the general case?

the unconstrained problem

can we do better than $1/3$ for the general case?

tight $1/2$ approximation for general non-negative submodular function

[Buchbinder et al.'15]

- deterministic algorithm, approximation $1/3$
- randomized algorithm, approximation $1/2$

deterministic algorithm for unconstrained submodular maximization

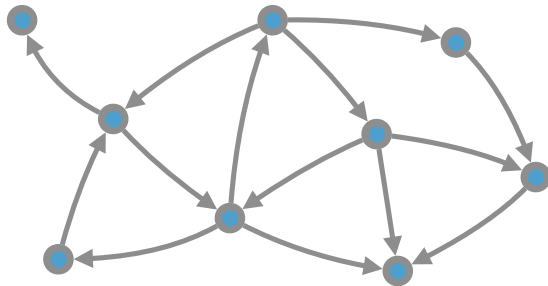
deterministicUSM [Buchbinder et al.'15]

1. $A_0 \leftarrow \emptyset, B_0 \leftarrow U$
2. for $i = 1, \dots, n$ % $n = |U|$
3. $\alpha_i \leftarrow f(A_{i-1} \cup \{u_i\}) - f(A_{i-1})$
4. $\beta_i \leftarrow f(B_{i-1} \setminus \{u_i\}) - f(B_{i-1})$
5. if $\alpha_i \geq \beta_i$ then
6. $A_i \leftarrow A_{i-1} \cup \{u_i\}, B_i \leftarrow B_{i-1}$
7. else
8. $A_i \leftarrow A_{i-1}, B_i \leftarrow B_{i-1} \setminus \{u_i\}$
9. return A_n % or equivalently B_n

example

input: directed graph $G = (V, E)$ with $|V| = n = 9$ vertices

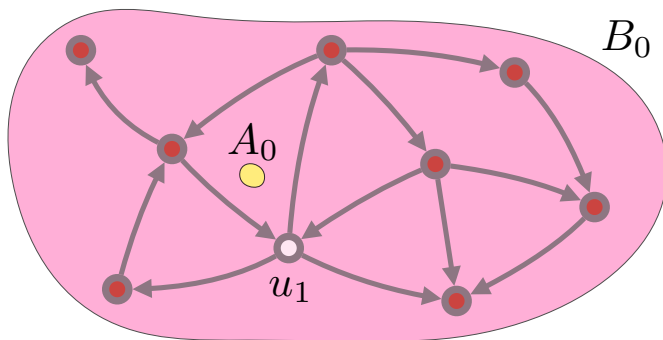
max di-cut: find $S \subseteq V$ to maximize $f(S) = \sum_{v \in S, u \in V \setminus S} e(v, u)$



example

input: directed graph $G = (V, E)$ with $|V| = n = 9$ vertices

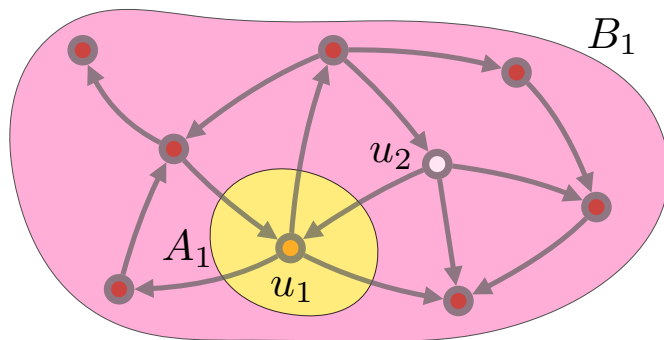
max di-cut: find $S \subseteq V$ to maximize $f(S) = \sum_{v \in S, u \in V \setminus S} e(v, u)$



example

input: directed graph $G = (V, E)$ with $|V| = n = 9$ vertices

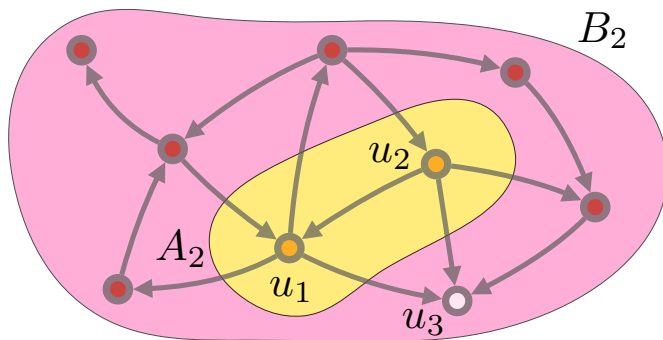
max di-cut: find $S \subseteq V$ to maximize $f(S) = \sum_{v \in S, u \in V \setminus S} e(v, u)$



example

input: directed graph $G = (V, E)$ with $|V| = n = 9$ vertices

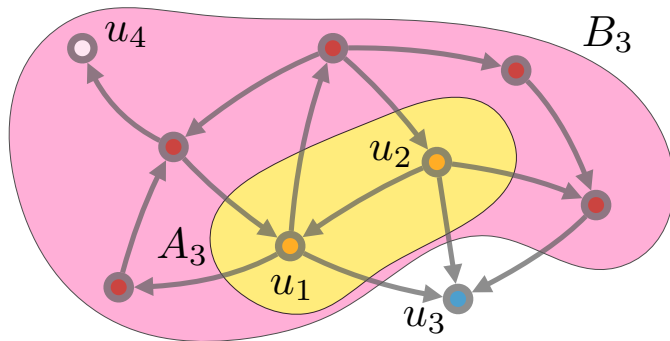
max di-cut: find $S \subseteq V$ to maximize $f(S) = \sum_{v \in S, u \in V \setminus S} e(v, u)$



example

input: directed graph $G = (V, E)$ with $|V| = n = 9$ vertices

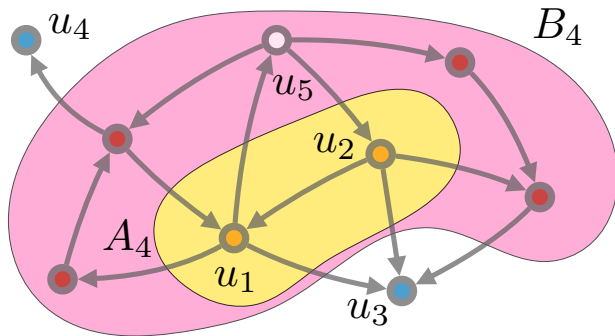
max di-cut: find $S \subseteq V$ to maximize $f(S) = \sum_{v \in S, u \in V \setminus S} e(v, u)$



example

input: **directed** graph $G = (V, E)$ with $|V| = n = 9$ vertices

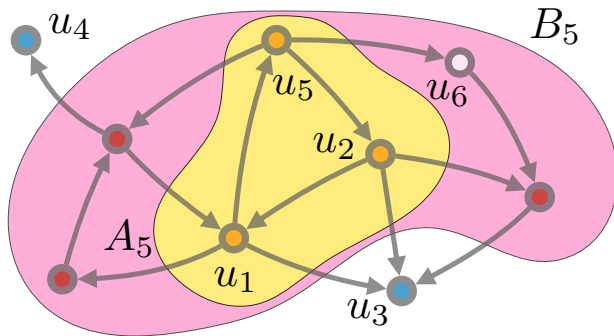
max di-cut: find $S \subseteq V$ to maximize $f(S) = \sum_{v \in S, u \in V \setminus S} e(v, u)$



example

input: directed graph $G = (V, E)$ with $|V| = n = 9$ vertices

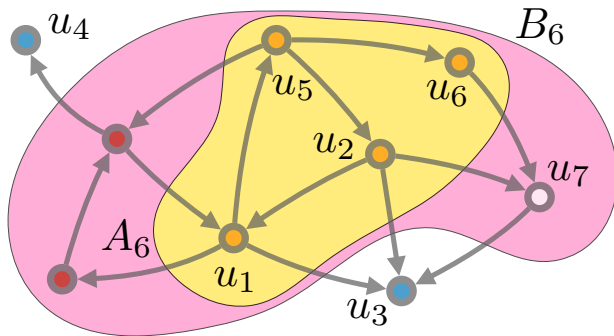
max di-cut: find $S \subseteq V$ to maximize $f(S) = \sum_{v \in S, u \in V \setminus S} e(v, u)$



example

input: directed graph $G = (V, E)$ with $|V| = n = 9$ vertices

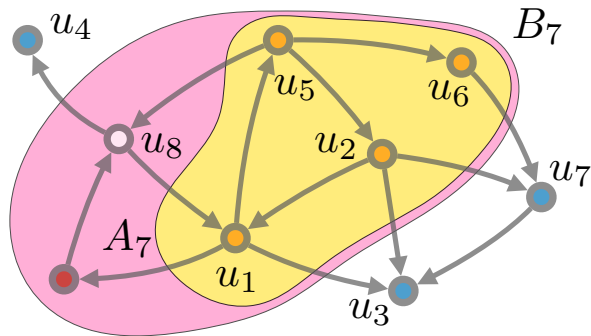
max di-cut: find $S \subseteq V$ to maximize $f(S) = \sum_{v \in S, u \in V \setminus S} e(v, u)$



example

input: **directed** graph $G = (V, E)$ with $|V| = n = 9$ vertices

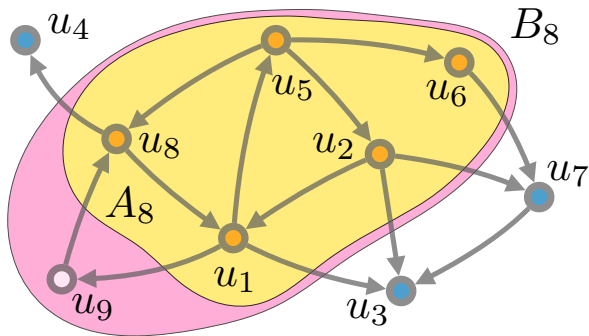
max di-cut: find $S \subseteq V$ to maximize $f(S) = \sum_{v \in S, u \in V \setminus S} e(v, u)$



example

input: directed graph $G = (V, E)$ with $|V| = n = 9$ vertices

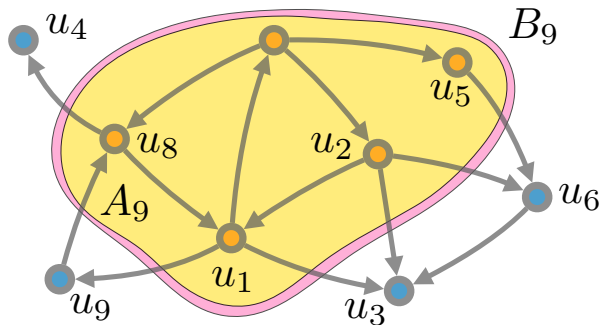
max di-cut: find $S \subseteq V$ to maximize $f(S) = \sum_{v \in S, u \in V \setminus S} e(v, u)$



example

input: directed graph $G = (V, E)$ with $|V| = n = 9$ vertices

max di-cut: find $S \subseteq V$ to maximize $f(S) = \sum_{v \in S, u \in V \setminus S} e(v, u)$



$$ALG = A_9 = B_9$$

Agenda

Submodular optimization

Maximization of submodular functions with cardinality constraints

Maximization of submodular functions under a matroid constraint

Maximization of non-monotone submodular functions

Max-sum diversification

max-sum diversification

[Borodin et al. '12]

given : a ground set U

a metric distance function $d : U \times U \rightarrow \mathbb{R}$ on U

and a submodular function $f : 2^U \rightarrow \mathbb{R}$

task : find $S \subseteq U$ such that

$\phi(S) = f(S) + \lambda \sum_{u,v \in S} d(u, v)$ is maximized and

$|S| \leq k$ (or other constraints, e.g., matroid constraints)

a diversity-aware version of submodular maximization

- we want to maximize $f(S)$ while also ensuring a diverse solution set S

max-sum diversification

[Borodin et al. '12]

- consider $S \subseteq U$, $x \in U \setminus S$, and define the following types of **marginal gain**

$$d_x(S) = \sum_{v \in S} d(x, v)$$

$$f_x(S) = f(S \cup \{x\}) - f(S)$$

$$\phi_x(S) = \frac{1}{2}f_x(S) + \lambda d_x(S)$$

- **greedy** algorithm on $\phi_x(S)$ gives $\frac{1}{2}$ -approximation for **cardinality constraint**
- **local-search** algorithm gives $\frac{1}{2}$ -approximation for arbitrary **matroid constraint**

conclusions

- maximization of submodular functions
- monotone, constraints, symmetric, ...
- recent developments in theory community
- simple algorithms
- neat analysis
- many applications in machine learning

Thank you!